Reflection: Multi-Agent Market Research & GTM Planning System (n8n vs. CrewAI)

VT_AGI: Applied Agentic AI -- Capstone (Product Strategy Simulation) Capstone Project

Created by Brock Frary

2026-08-06

1. Project Overview

This capstone implements a multi-agent workflow that automates market research and go-to-market (GTM) planning, and builds it twice -- once in n8n, once in CrewAI -- so the two implementations can be compared head to head on cost, latency, and reliability. Four agents are shared across both: a Head Planner that drafts the research brief, a Research Agent that gathers cited findings via two distinct tools (SerpAPI for search, a custom-built MCP server for fetch/citation), an Analyst Agent that synthesizes findings into a competitor comparison, and a Strategy Agent that drafts the final GTM plan -- target customer, value proposition, messaging/channels, and a launch outline. n8n additionally exports the plan to a real Google Doc; CrewAI persists each stage's output to markdown files. Both implementations were run end to end against the same fixed test brief (a budget noise-cancelling headphone for commuters) across 15 logged runs, with real cost, latency, and human-rubric-quality data captured for the comparison, not simulated or estimated figures.

2. Architecture

Both implementations wire the same four agents to a shared tool layer: a custom MCP server (search, fetch, citation/caching, served over SSE) and SerpAPI, both reachable identically from n8n's native MCP Client Tool node and CrewAI's MCPServerAdapter. The two diagrams below present the same system from two angles -- the shared end-to-end workflow every brief passes through, and a swimlane that makes explicit where n8n and CrewAI diverge (orchestration engine, final export format) and where they share real infrastructure (the MCP server, SerpAPI, and the run-log schema both write to).

Figure A -- Shared Four-Agent Workflow

The full pipeline from project brief through Head Planner, Research Agent (with its two tool integrations), Analyst Agent, and Strategy Agent, to the implementation-specific export/persist step. Every agent writes a run-log row (cost, tokens, latency) to the shared comparison/run_logs/run_logs.jsonl schema, regardless of which implementation produced it.

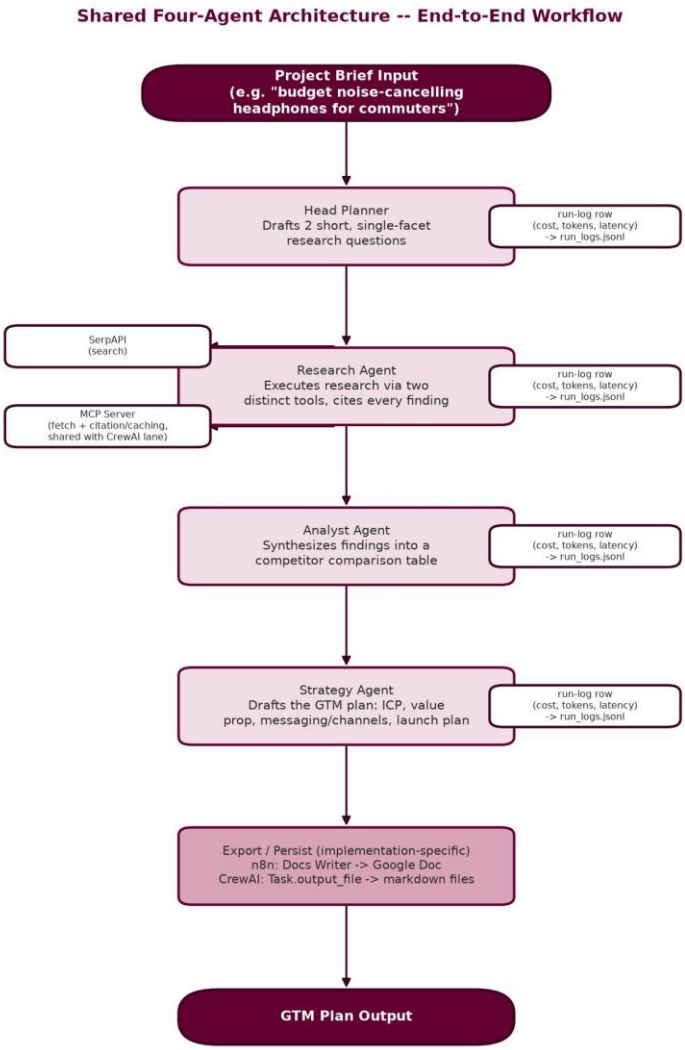
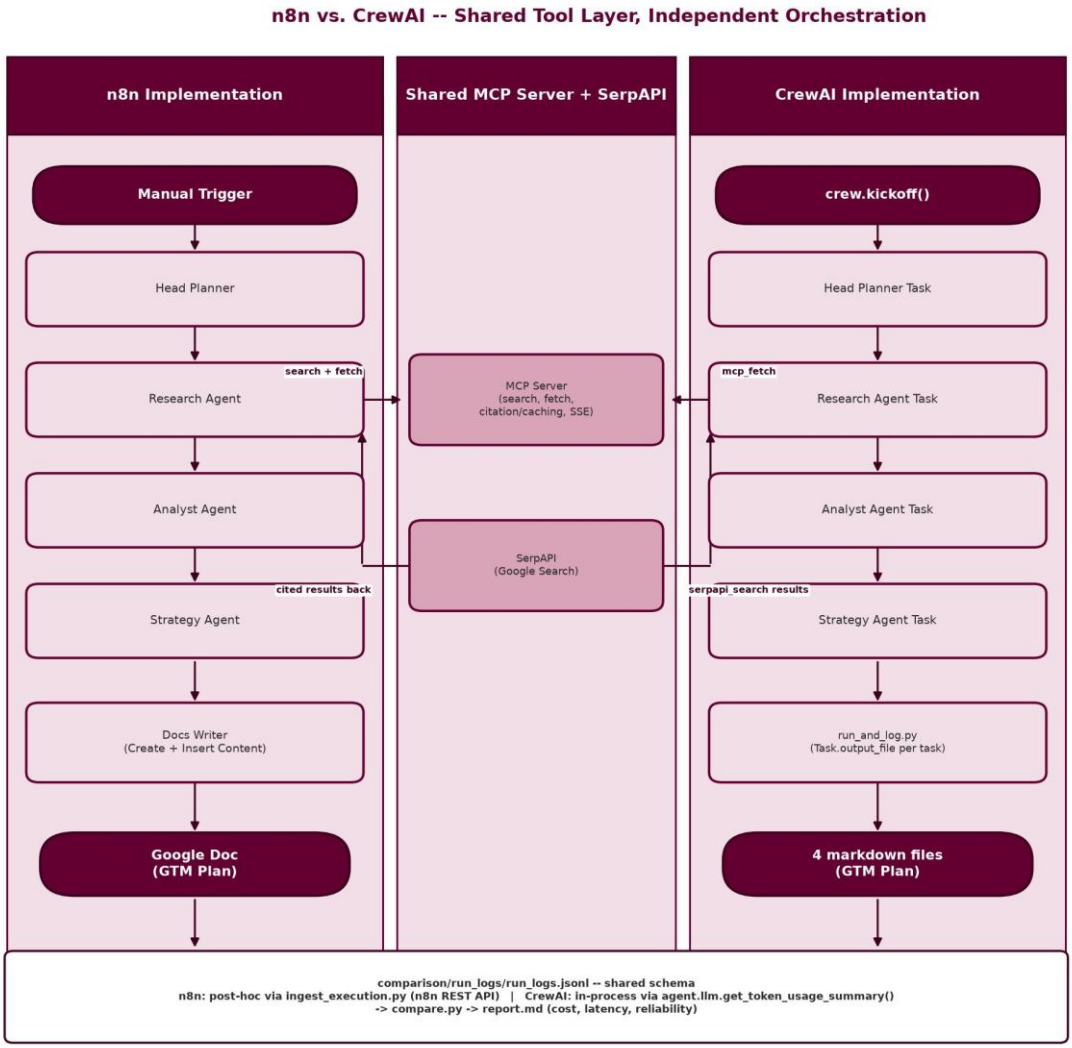


Figure B -- n8n vs. CrewAI Swimlane

The same system viewed as three lanes: n8n's orchestration, CrewAI's orchestration, and the shared MCP Server + SerpAPI tool layer both Research Agent stages call into independently. Both lanes terminate in a different artifact (a Google Doc vs. four markdown files) but converge on the same run-log schema at the bottom, which is what makes an apples-to-apples cost/latency comparison possible at all.



3. Design Decisions

Build it twice -- n8n and CrewAI, not a single choice

The assignment's core ask was a comparison, not a single best implementation, so both were built to the same agent spec from the start rather than picking one and reasoning about the other in the abstract. This meant every design decision below had to hold up identically in a low-code visual tool and a code-first Python framework, which surfaced real asymmetries (see Section 4) that a single-implementation project would never have found.

Custom MCP server, adapted rather than built from scratch

The assignment requires an MCP server but not that it be written from nothing. Rather than build blind on a first-time protocol or gamble on an unverified off-the-shelf bundle, the official reference fetch server (modelcontextprotocol/servers) was adapted and extended with a custom search tool (SerpAPI-backed) and a citation/caching layer, served over real MCP-over-SSE. This was the lowest-cost path that still taught real MCP mechanics through a working example, and it paid off directly: the exact same running server is consumed natively by both n8n's MCP Client Tool node and CrewAI's MCPServerAdapter, with no protocol-specific glue code on either side.

A standalone log_server instead of routing around n8n's sandbox

Both implementations need to persist per-run cost/latency/token data somewhere the comparison script can read without special-casing either one. n8n's Code node sandboxes require('fs'), and two 2026 CVEs (CVE-2026-1470, CVE-2026-0863) in n8n's sandbox-escape surface made routing around that sandbox the wrong move rather than a shortcut. The resolution was a small standalone Python process (comparison/log_server, stdlib http.server only, zero new dependencies) that both implementations write to through a shared schema -- n8n via a POST /log HTTP Request node, CrewAI by importing the same validation/append logic directly in-process, since it is already Python.

CrewAI's max_iter set to the framework default, not a lower number

n8n's tool-using agents cap at maxIterations: 5. The instinct was to mirror that directly on CrewAI's equivalent agents, but checking real prior runs first showed the Research Agent using up to 18-20 tool-call iterations in normal, successful operation -- a cap of 5-10 would have silently truncated already-verified-working behavior. max_iter was set to 25 (CrewAI's own framework default) instead, made explicit for documentation parity with n8n's setting rather than left implicit, but not lowered just to mirror n8n's number superficially.

4. Challenges and Resolutions

n8n does not expose per-agent token usage

n8n's AI Agent node has no way to read its own token usage from within the workflow (a confirmed platform limitation, n8n-io/n8n#26302), so an in-workflow logging node cannot report real cost data as it happens. Resolved with ingest_execution.py, a post-hoc script that calls n8n's own REST API after a run completes -- which does have the real numbers -- and cross-references them against the exported workflow JSON to attribute each Chat Model's usage to the correct agent. CrewAI has no equivalent problem: agent.llm.get_token_usage_summary() returns real per-agent data in-process, immediately after kickoff() -- a genuine, structural observability gap between the two platforms, not just a difference in how this project happened to build each one.

A stuck tool-call loop from two overlapping search tools

Early on, the MCP server's own search tool and the native SerpAPI node were both attached to n8n's Research Agent, giving the model two overlapping ways to search the web. It kept alternating between them instead of concluding, burning roughly 200,000 to 270,000 tokens per run before hitting the iteration cap. The fix was giving each tool exactly one distinct job instead of removing either one: MCP's Tools to Include narrowed to fetch-only, SerpAPI left as the sole search path. This also matches the rubric's own phrasing more literally -- MCP tools for research and SerpAPI for queries.

Source volatility -- robots.txt blocks and a real API timeout

Both implementations hit genuine, unstaged failures during real runs: a 403 Forbidden and a robots.txt disallow on two different fetch targets in n8n, and a real SerpAPI read timeout in CrewAI. None were coded exceptions to catch and retry automatically -- each was resolved at the prompt level, telling the Research Agent to move on to a different result rather than retry a blocked URL. All three recovered cleanly with no manual intervention, across different failure types, confirming the fallback generalizes rather than only covering the one case it was first written for.

A node-config gotcha that cost real debugging time

n8n's AI Agent node has a Source for Prompt (User Message) setting that is a dropdown (promptType: auto vs. define), separate from the actual prompt text field underneath it. An early attempt typed a named-node expression directly into the dropdown's own toggle instead of selecting define first, which silently left the real prompt field

empty and produced a persistent "Prompt (User Message) is required" error with no obvious cause in the error text itself. Once diagnosed, this became a standing check applied to every subsequent agent node rather than something re-discovered per node.

A full rubric audit near the end, not just trusting self-reported progress

With all five build phases marked complete in the project's own working notes, a full line-by-line audit was run against the actual grading rubric document rather than trusting those self-reported checkmarks. It found a stale top-of-README status banner, a Risk & Mitigations table that claimed mitigations (exponential backoff, automated uncited-claim flagging, post-write formatting validation) that were never actually coded, and CrewAI's interim task outputs never being persisted to files despite the rubric explicitly asking for retained interim outputs. None were large engineering problems, but all four were the specific kind of gap that stays invisible if "done" is only checked against a task list instead of against what the artifact actually does.

5. Trade-offs

The following table summarizes the key trade-offs made during the build:

Decision	Trade-off
Build both n8n and CrewAI to the same spec	Real, apples-to-apples comparison data; roughly double the build and testing surface of picking one
Adapted MCP reference server vs. built from scratch	Faster, lower-risk path to real MCP mechanics on a first-time protocol; less control over the fetch tool's internals than a from-scratch build
Standalone log_server vs. n8n Code-node file I/O	Sidesteps n8n's fs sandbox and two known CVEs cleanly; one more small service to run locally alongside n8n and the MCP server
n8n maxIterations: 5 vs. CrewAI max_iter: 25	Each tuned to its own platform's real observed behavior rather than forced to match; the two numbers are not directly comparable at face value without that context
Prompt-level fetch-failure fallback vs. coded retry/backoff	Cheap, one line per agent, proven to generalize across failure types in real runs; behavior depends on the model actually following the instruction rather than a guaranteed code path
n8n token usage via post-hoc REST ingestion vs. CrewAI's in-process read	Both produce real per-agent numbers in the end; n8n's path is a platform-forced extra script CrewAI simply never needed

6. Conclusion

The project delivers two working end-to-end implementations of the same four-agent GTM planning system, verified across 15 real runs with a 100% run-success rate on both sides. n8n averaged $0.0642 and 1m 59.4s per run; CrewAI averaged $0.1379 and 6m 39.0s per run -- both comfortably under the $0.50/run budget cap, with n8n roughly 2.1x cheaper and considerably faster on average, largely a function of n8n's lighter-weight agent loop versus CrewAI's more deliberate multi-iteration tool reasoning. Human rubric review scored both implementations an average of 4.33/5, each strongest on a different criterion (n8n on differentiation, CrewAI on feasibility) rather than one being unambiguously better. The honest gap, caught during a full rubric audit rather than assumed away: several risk mitigations were described in early documentation more confidently than they

were actually implemented, and CrewAI's interim outputs were not persisted to files until that audit caught it -- both corrected before submission, and both worth naming rather than smoothing over.

7. Personal Reflections

n8n vs. CrewAI, hands-on

Building the same system twice made the trade-off between the two platforms concrete instead of theoretical. n8n's visual canvas made the agent chain easy to see and debug at a glance -- watching a run's green checkmarks propagate node by node is a genuinely good debugging experience. But it has a real blind spot: it cannot tell you what a run actually cost while it's running, or even easily after, without going around it via its own REST API. CrewAI is the opposite experience -- less visual, more code, but it will hand you exact per-agent token usage the moment a run finishes, no extra script required. Neither platform is simply better; they made different bets about what should be easy, and this project ended up needing to work around n8n's bet more than CrewAI's.

Building an MCP server for the first time

This was the first time working with the Model Context Protocol directly, and adapting the official reference server rather than building from a blank file was the right call for a first attempt -- it kept the real protocol mechanics (the SSE transport, the tools/list and tools/call handshake) visible without also debugging basic HTTP plumbing from scratch. The payoff was worth it: seeing n8n's MCP Client node and CrewAI's MCPServerAdapter both talk to the exact same running server, with no per-implementation glue, made MCP's interoperability promise feel real rather than like a spec on paper.

Auditing the rubric instead of trusting my own checklist

The most valuable single thing done late in this project was going back to the actual grading rubric, line by line, instead of trusting a project roadmap that already said every phase was complete. It caught real gaps -- documentation that claimed more than was built, an unpersisted output the rubric explicitly asked for -- none of which would have surfaced from re-reading my own notes, because my own notes were the thing that was wrong. Worth doing on every project going forward, not just the ones with a formal rubric to check against.

On the environment

Running n8n, the MCP server, and CrewAI all inside the same WSL2 Ubuntu instance avoided most of the Windows/WSL2 networking friction a prior course project had already run into the hard way -- the localhost-vs-host-IP issue and the interactive-terminal-for-PATH issue never came up here, precisely because they were already known going in. That's a small, honest example of a lesson actually staying learned from one project to the next.

8. Build Walkthrough -- Screenshots

The following screenshots document Phases 0, 2, and 3 of the build -- environment/access setup, the n8n implementation, and the CrewAI implementation -- in sequence. Each image corresponds to an entry in the project's SCREENSHOTS.md inventory. Phases 1 (MCP server), 4 (testing), and 5 (comparison) are covered in the narrative sections above rather than with dedicated screenshots for this pass.

Figure 1 -- Google Cloud Project Created

The Google Cloud Console "Welcome" page confirming project vt-capstone-gtm-planner exists and is selected as active, the first step toward Google Docs export.

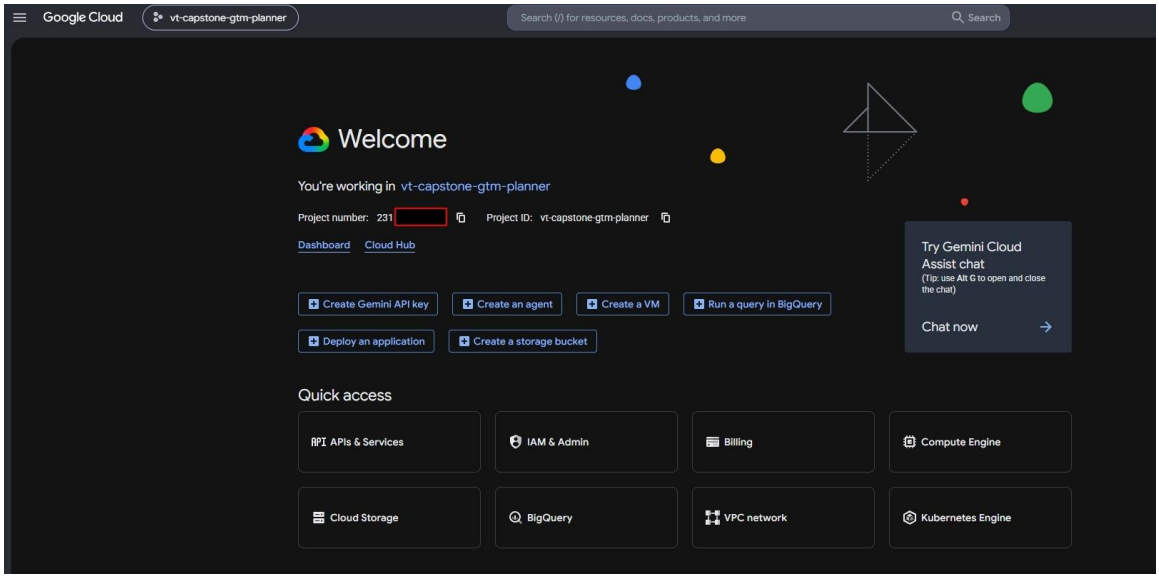


Figure 2 -- Google Docs API Enabled

The API/Service Details page for docs.googleapis.com, status Enabled -- required for the Docs Writer node to create and edit documents.

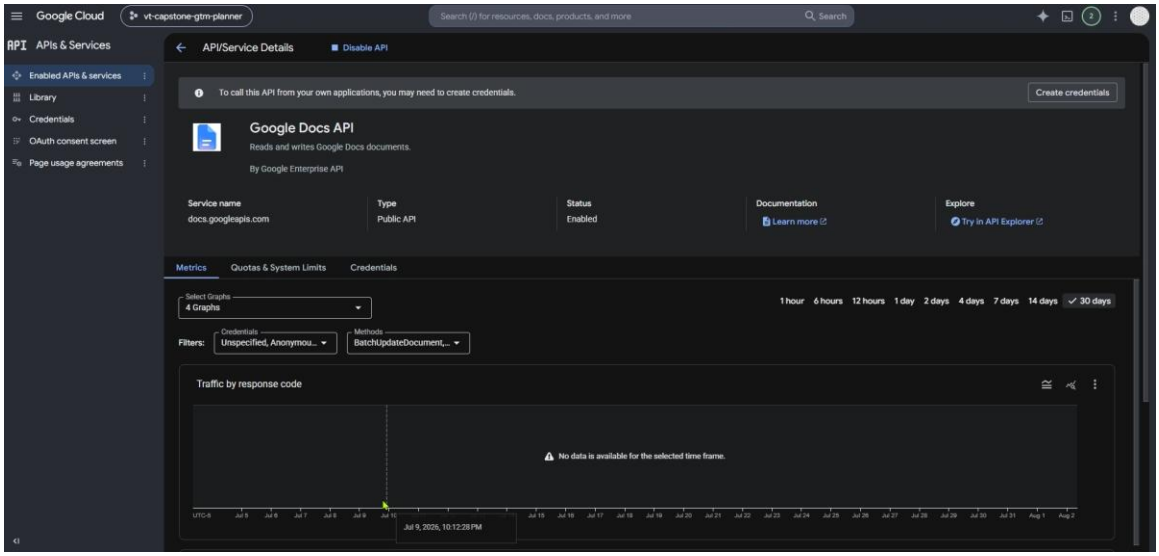


Figure 3 -- Google Drive API Enabled

The API/Service Details page for drive.googleapis.com, status Enabled -- required alongside Docs API for file-level access to the created document.

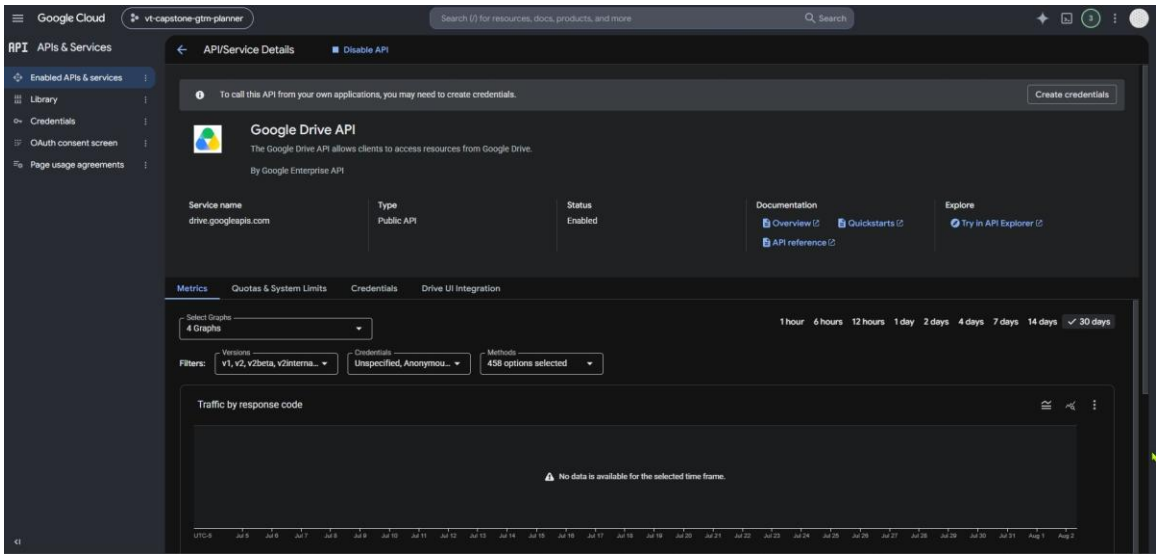


Figure 4 -- OAuth Branding Created

The Google Auth Platform Overview page immediately after the branding/consent-screen step, before any OAuth Client ID existed yet.

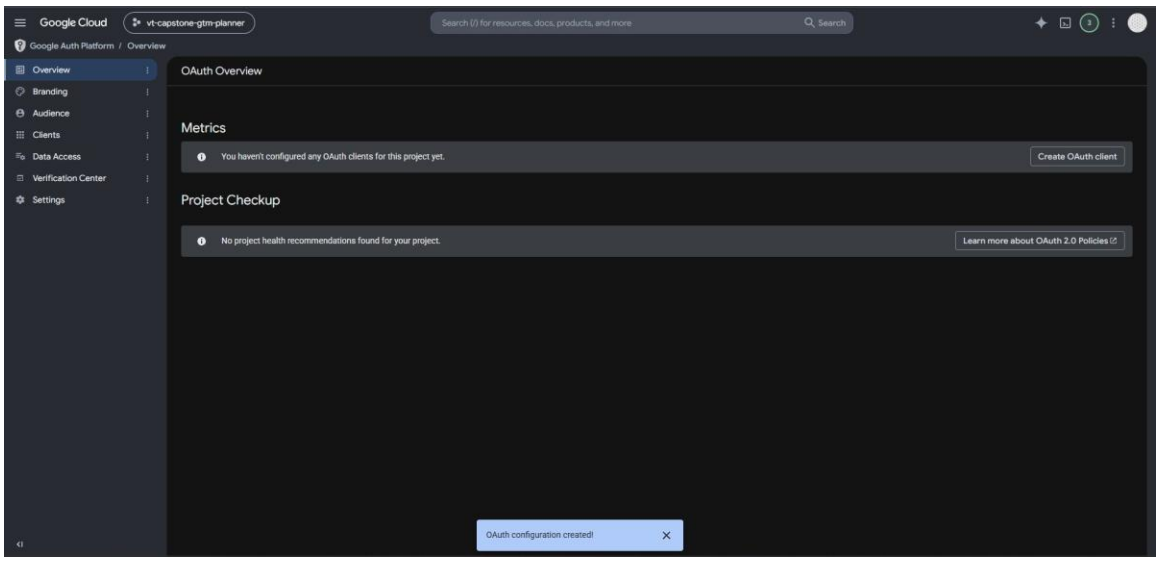


Figure 5 -- OAuth Scopes Configured

The Data Access page showing both scopes required for Docs export: the non-sensitive drive.file scope and the sensitive documents scope. No restricted scopes were added.

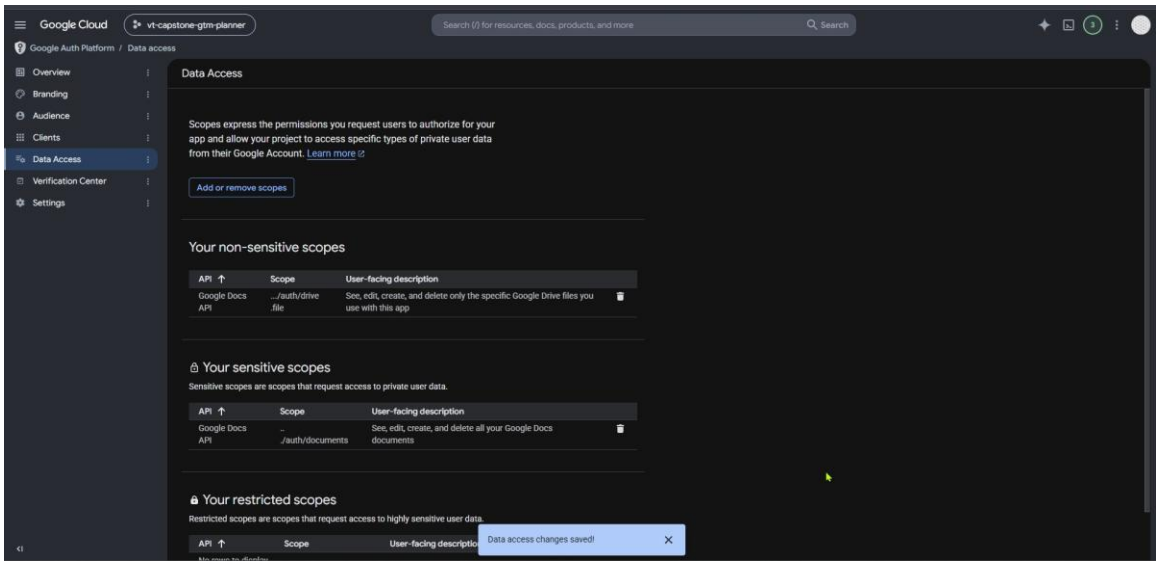


Figure 6 -- OAuth Test User Added

The Audience page confirming the app is in Testing mode (External) with the one required test user added, under the 100-user testing cap.

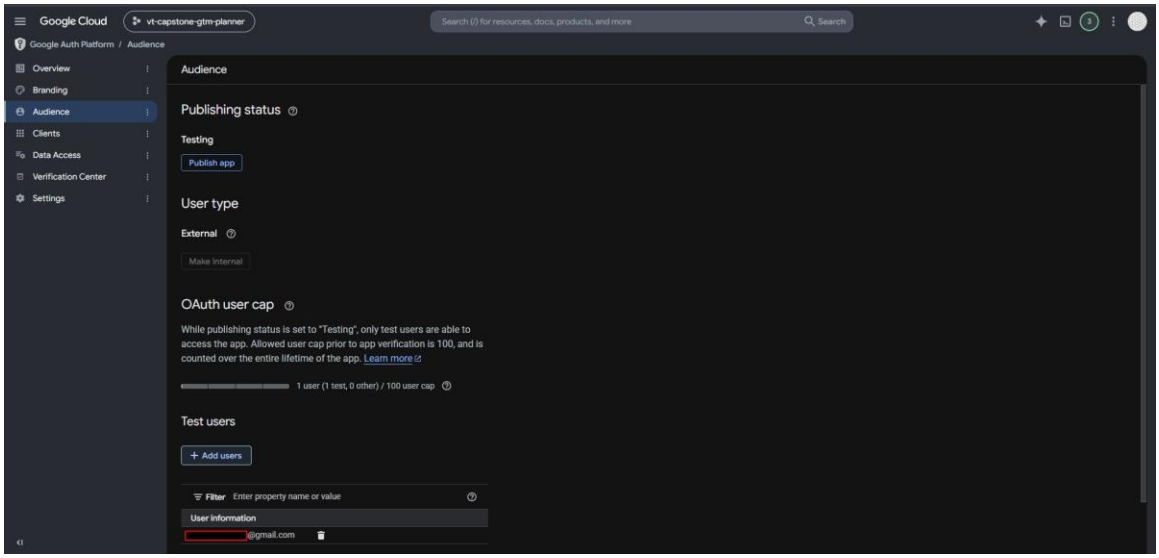


Figure 7 -- OAuth Client Created

The detail view of the "n8n local" Web application OAuth 2.0 Client ID, redirect URI set to n8n's local OAuth callback, status Enabled.

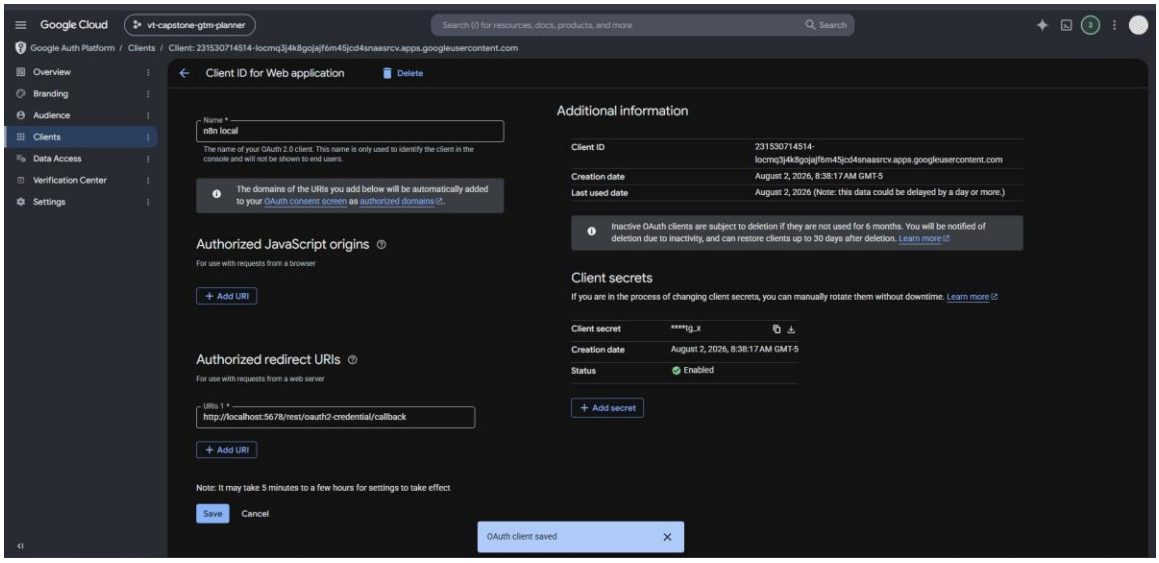


Figure 8 -- MCP Client Connectivity Confirmed (n8n)

n8n's editor showing a minimal Trigger -> AI Agent -> MCP Client Tool run against example.com. The MCP Client node's output returned a cited, timestamped snapshot (citation_id, source_tool, fetched_at), confirming the MCP server is reachable from n8n and the Phase 1 citation design works end to end, driven by the agent's own tool choice rather than a forced call.

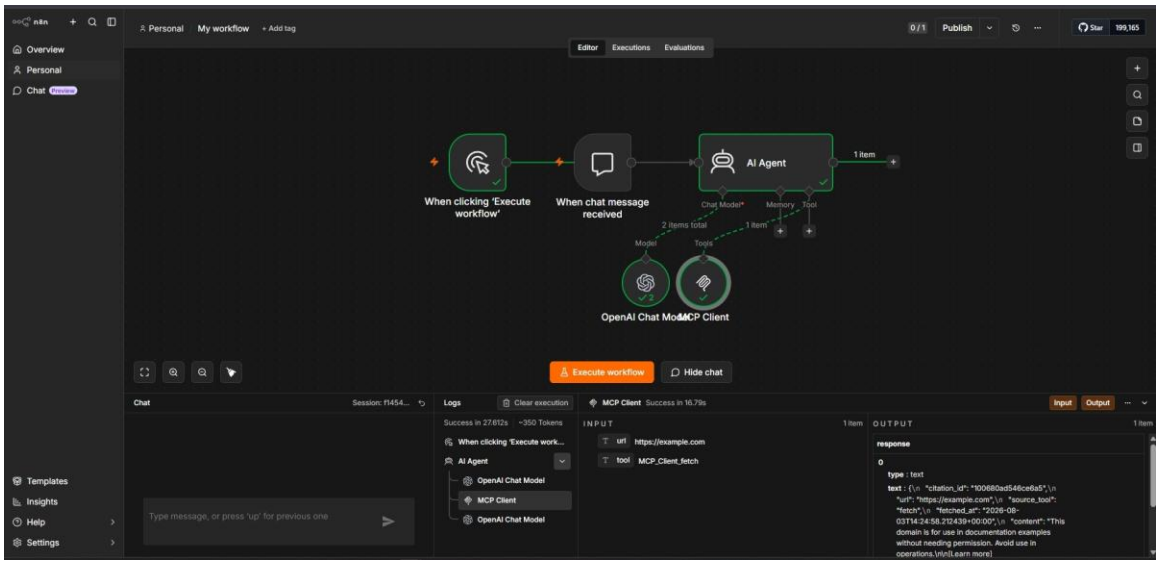


Figure 9 -- Research Agent: SerpAPI + MCP Working Together

The Research Agent node's output after fixing a stuck tool-call loop (MCP's own search tool and the native SerpAPI node had overlapping jobs, causing the model to alternate between them instead of concluding). With MCP narrowed to fetch-only and SerpAPI as the sole search tool, this run completed in 3 iterations (well under the Max Iterations cap of 5) with a clean search -> fetch -> cite chain.

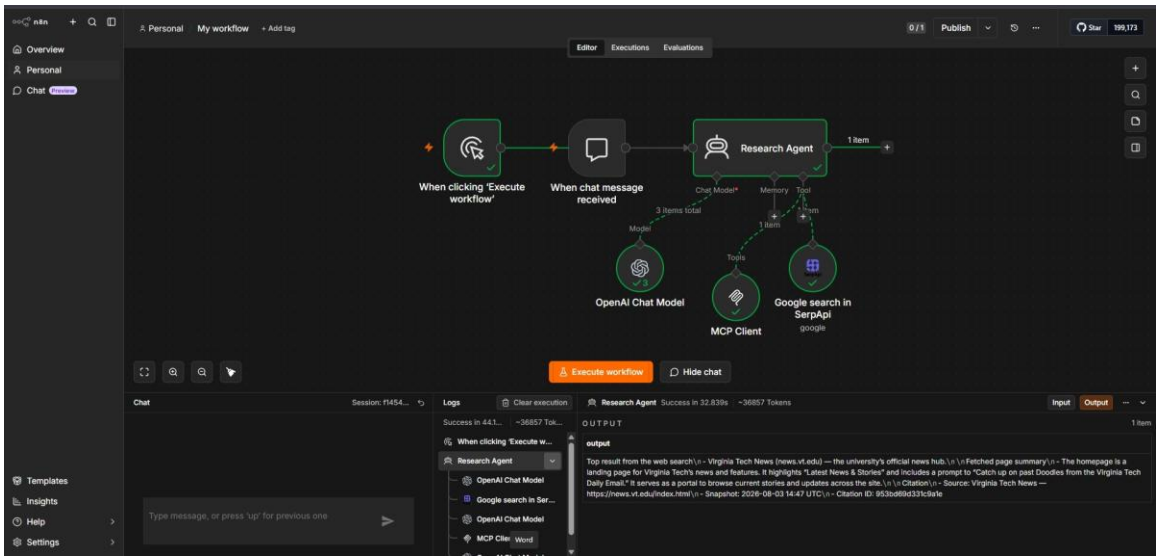


Figure 10 -- Head Planner to Research Agent Hand-off

The first successful two-agent run on the real, locked-in test brief (a budget noise-cancelling headphone for commuters). Head Planner produced exactly 2 single-facet research questions; Research Agent executed both without context-window overflow, after scope-control fixes replaced an earlier "production-ready" 6-category brief that had fragmented into 18-26+ tool calls and blown past the model's context window.

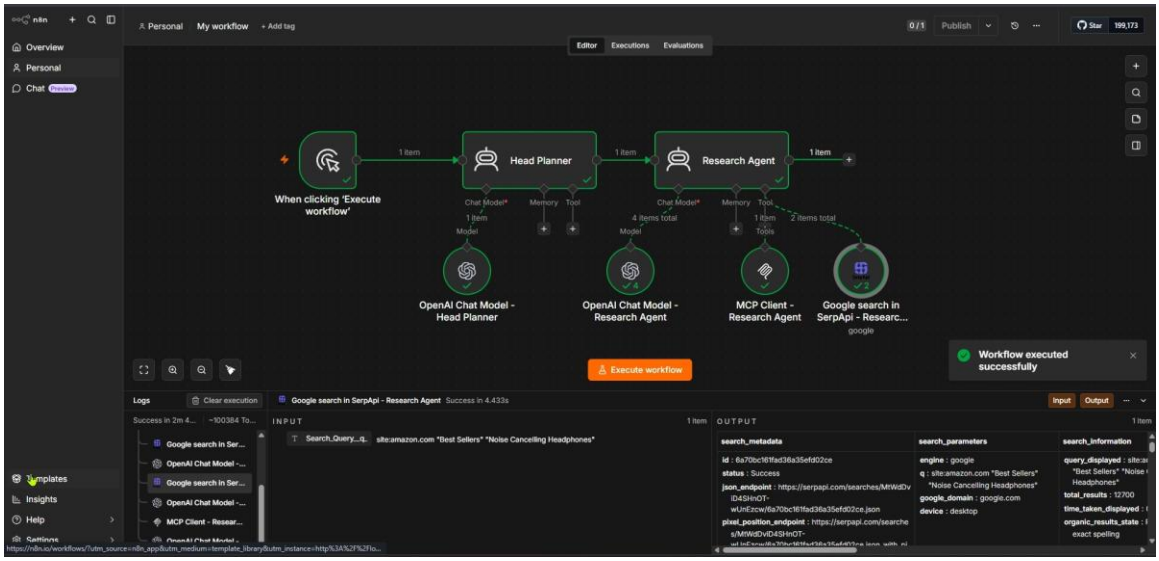


Figure 11 -- Analyst Agent Synthesis, Including a Real Recovery

The first three-agent chain run. Analyst Agent produced a specific competitor comparison genuinely derived from Research Agent's findings. This run also hit a real 403 Forbidden fetch failure mid-run; Research Agent's fetch-failure fallback instruction ("move on to a different result, don't retry") recovered without any manual intervention.

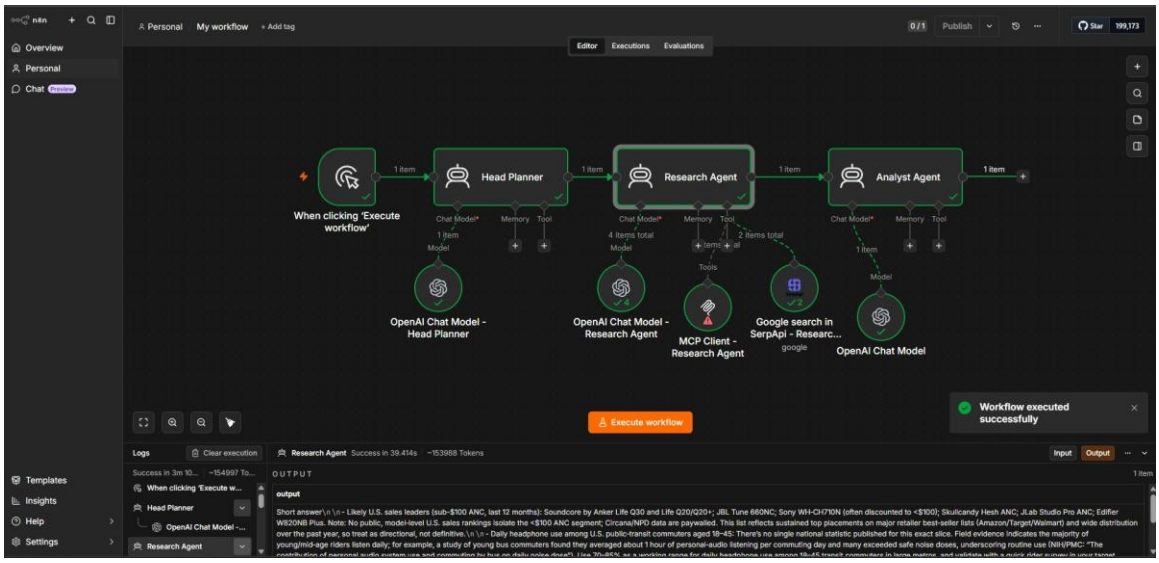


Figure 12 -- Strategy Agent Drafts the GTM Plan

The first five-node chain run through Strategy Agent, producing a concrete ICP statement, value proposition, messaging/channel suggestions, and a launch-plan outline, all genuinely tied to the upstream Analyst Agent synthesis rather than generic boilerplate.

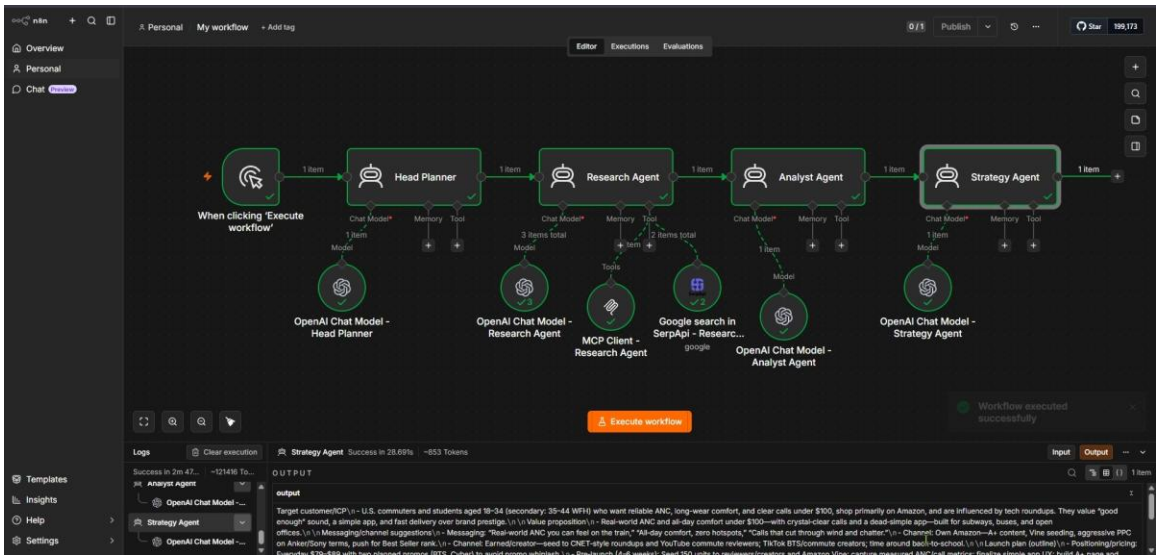


Figure 13 -- Docs Writer Creates the Document

The six-node chain extended through Docs Writer's Create operation, producing a blank titled Google Doc. This run hit a different fetch-failure type (robots.txt disallows fetching) than Figure 11's 403 -- the same fallback instruction handled it without any prompt changes, confirming the fix generalizes across failure reasons.

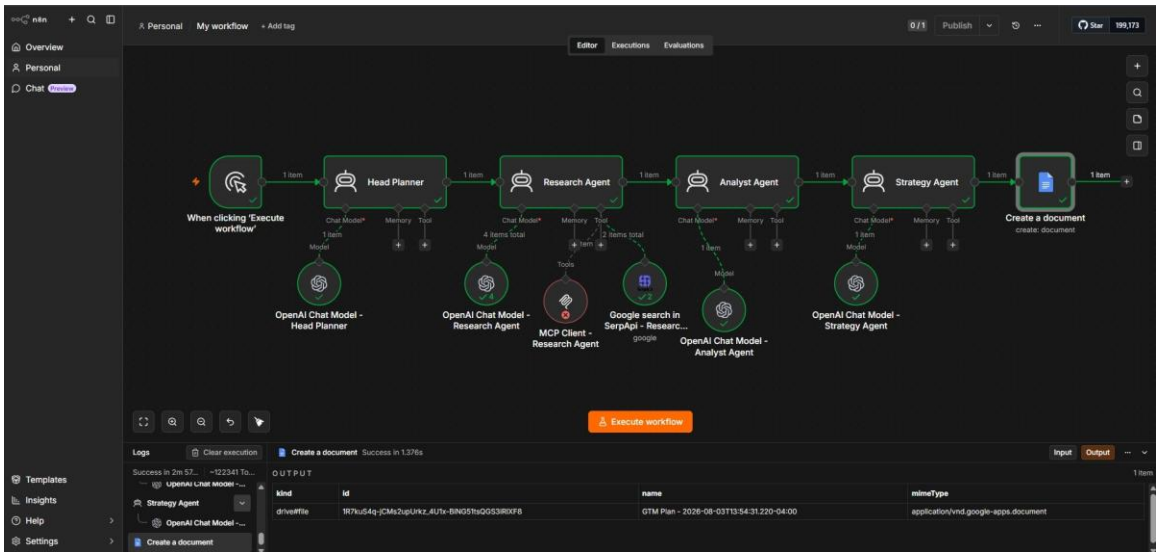


Figure 14 -- Document Confirmed in Google Drive

The Google Drive listing showing the same document from Figure 13 by name and timestamp, cross-confirming the OAuth2 credential (drive.file + documents scopes only) correctly creates real files in the authenticated account.

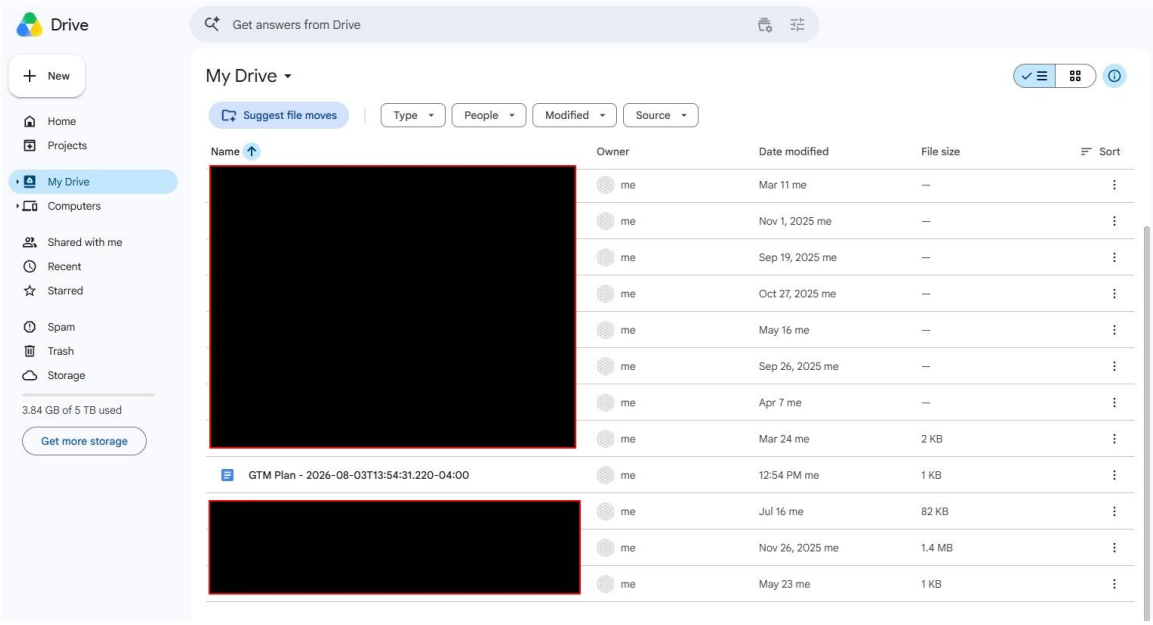


Figure 15 -- Final GTM Plan Document Content

The opened Google Doc showing the real inserted GTM plan text with genuine paragraph breaks (not literal \n characters) -- Target customer/ICP, value proposition, messaging/channels, and a launch plan. The end product of the full seven-node n8n chain.

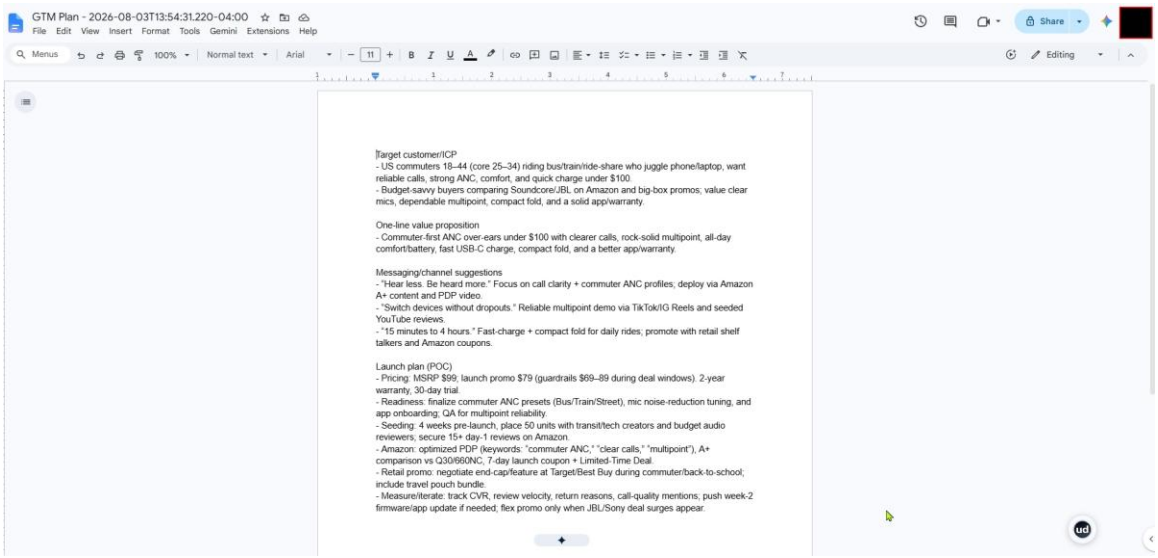


Figure 16 -- Full n8n Chain, Single-Click End to End

A single Execute Workflow run completing all seven nodes in one pass, 3m 5s total -- well inside the rubric's 15-minute target -- including graceful recovery from the same robots.txt fetch block seen in Figure 13, with no manual intervention.

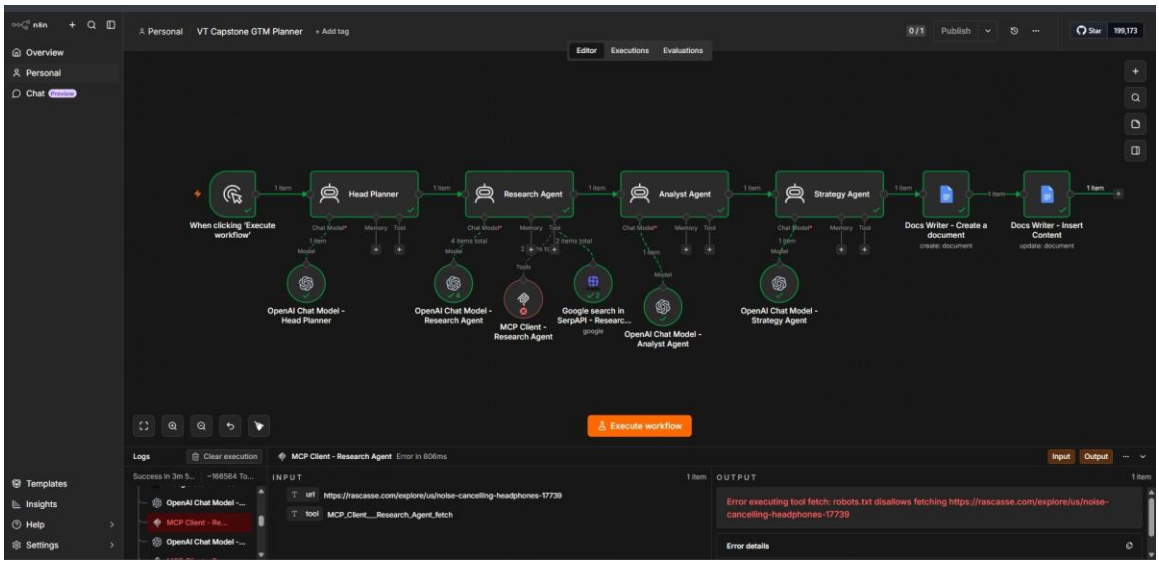


Figure 17 -- CrewAI: Head Planner Task Completes

The Crew Execution Started panel through Head Planner's Task Completion, from a real run driven by run_and_log.py's plain terminal output. Head Planner produced the same two-question research brief structure as the n8n implementation, on the identical fixed test brief used throughout this comparison.

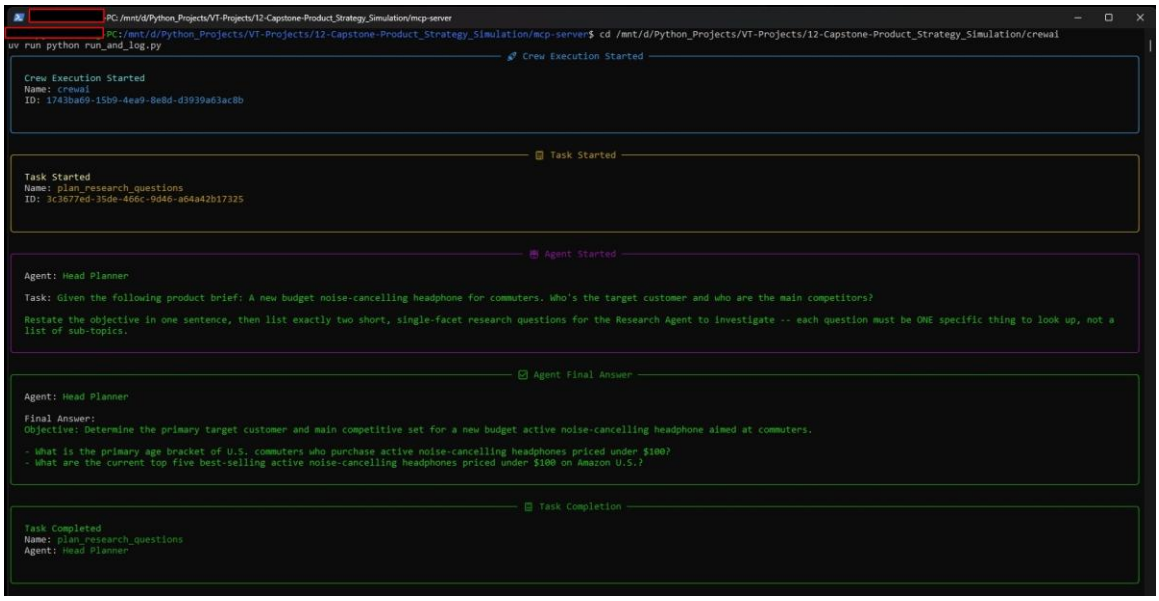


Figure 18 -- CrewAI Research Agent: Real SerpAPI Timeout Recovery

A genuine transient failure -- serpapi.com read timed out on Tool Execution #1 -- followed by the agent's own reasoning loop moving on to a reworded query with no manual intervention, the same source-volatility risk category the n8n implementation also had to handle.

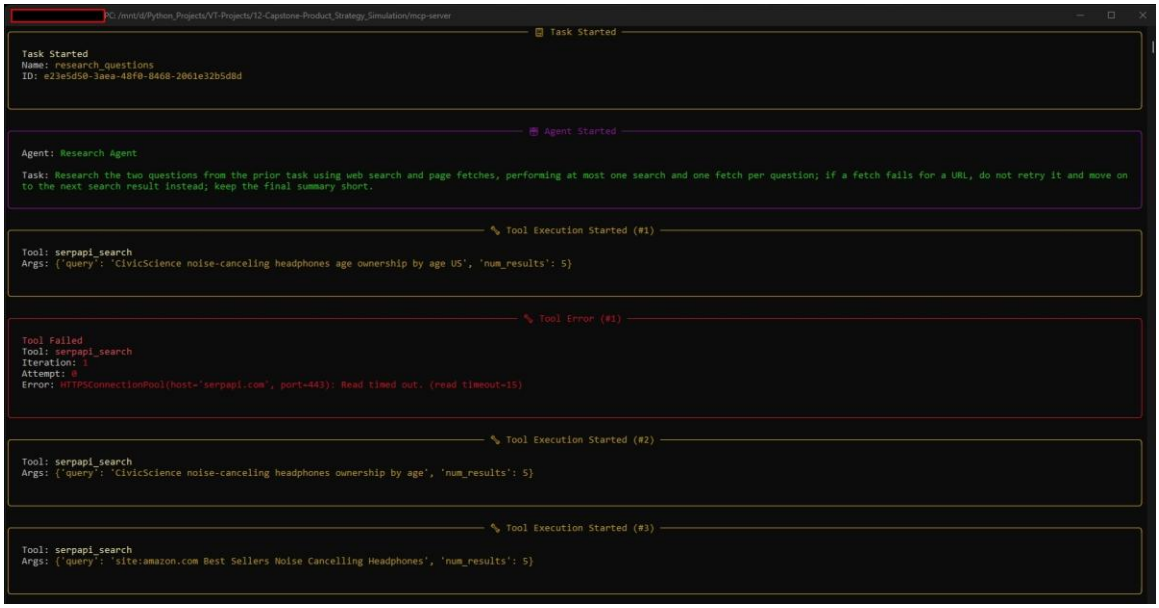


Figure 19 -- CrewAI Research Agent: MCP Fetch, Blocked and Successful

Both a real robots.txt block and a real successful fetch through this project's own MCP server, the successful one returning a full citation object (citation_id, url, source_tool, fetched_at) -- confirming the Phase 1 citation design works from CrewAI, not just n8n.

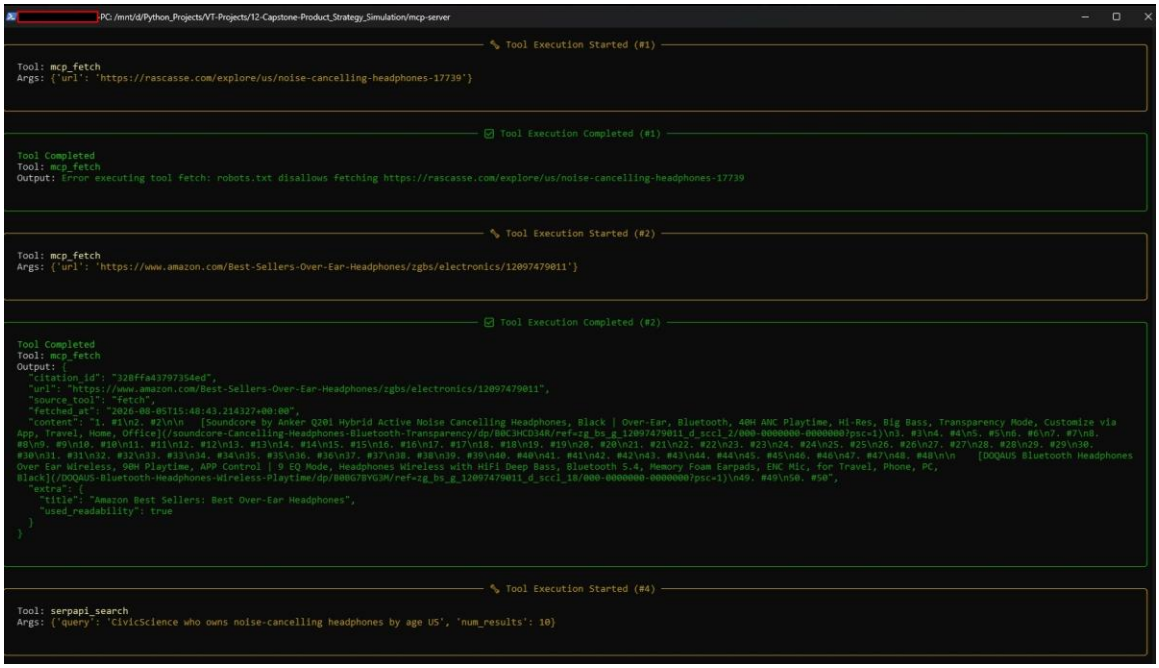


Figure 20 -- CrewAI Research Agent: Cited Final Answer

The tail of Research Agent's Final Answer, listing a concise finding, cited source, and full content snapshot per research question -- including the agent's own honest caveat that one source page was dynamic and only partially captured, rather than overstating confidence.

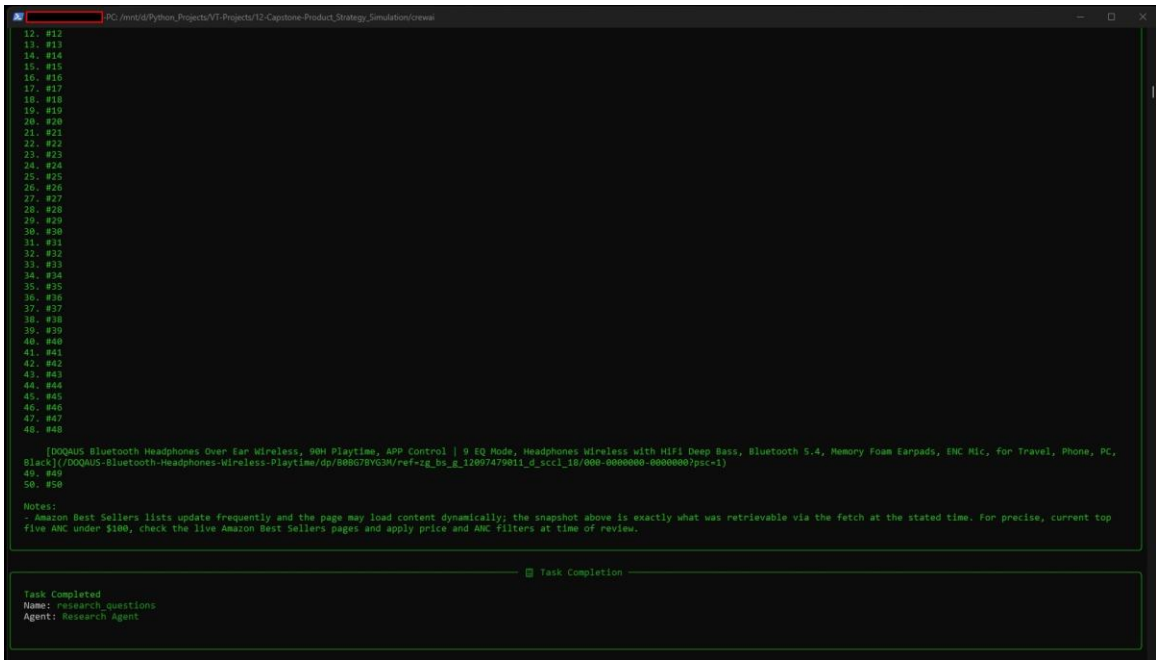


Figure 21 -- CrewAI Analyst Agent Synthesis

A markdown competitor comparison table across five real products (Soundcore Life Q20i/Q30, JBL Tune 660NC, TOZO HT2/HT6, Srhythm NC25/NC35) plus a qualitative "Competitive reality" paragraph, genuinely derived from Research Agent's findings, mirroring n8n's Figure 11 proof point.

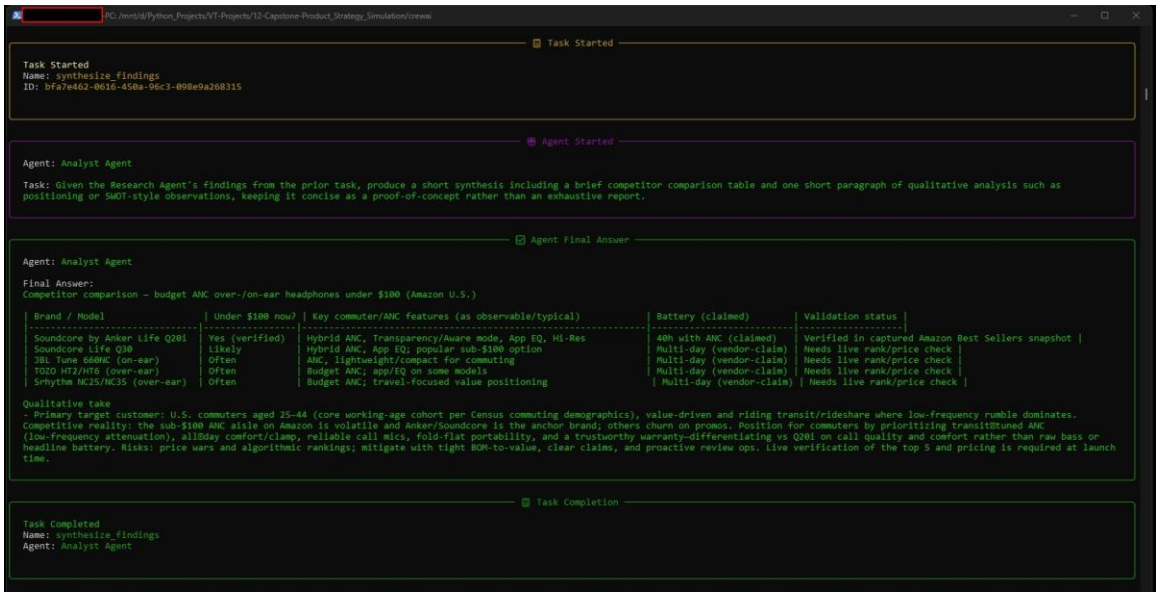


Figure 22 -- CrewAI Strategy Agent GTM Plan

An ICP statement, value proposition, three messaging/channel suggestions, and a phased launch-plan outline (pre-launch, weeks 1-4, days 30-60) -- CrewAI's independently drafted equivalent to n8n's Figure 12 output, from the same upstream synthesis pattern.

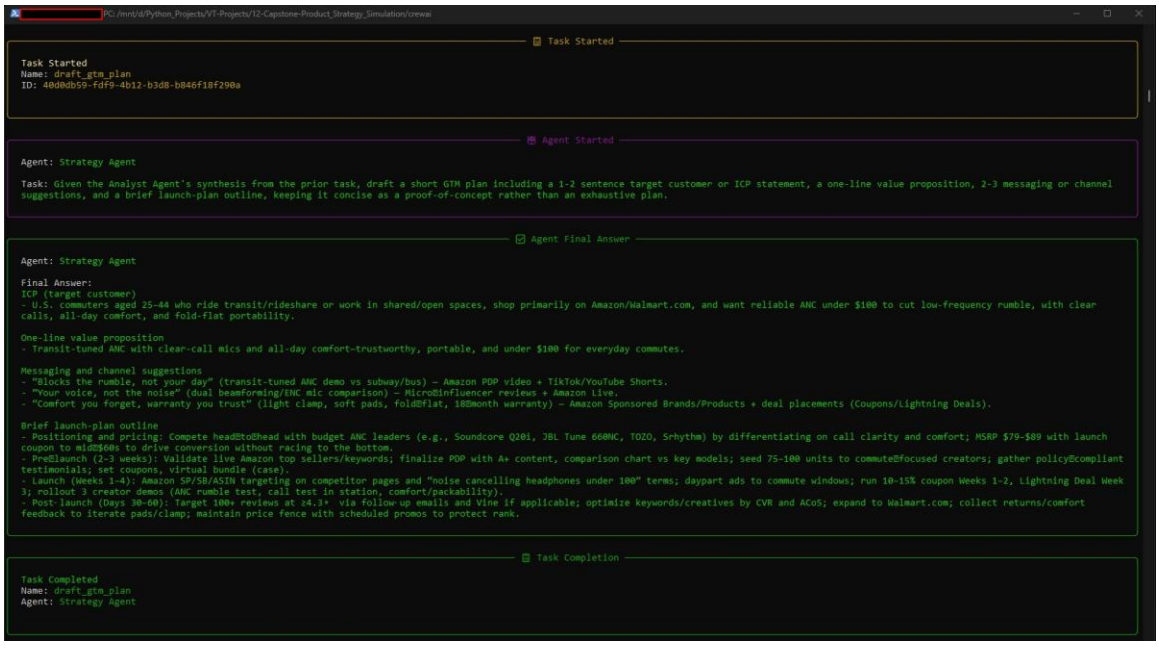


Figure 23 -- CrewAI Crew Completion

The Crew Completion panel and run_and_log.py's printed FINAL RESULT block, confirming the entire four-agent chain completes end to end in a single script invocation and the crew's return value matches Strategy Agent's actual Final Answer.

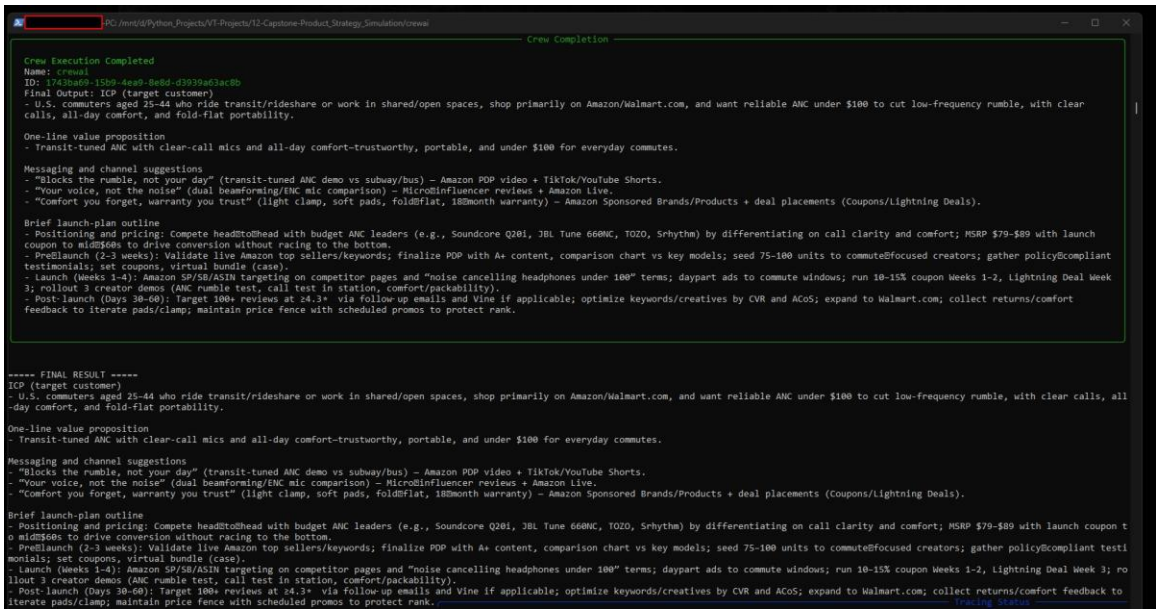


Figure 24 -- CrewAI Run-Log Confirmation

run_and_log.py's own [INFO] summary: 4 schema-valid rows appended to comparison/run_logs/run_logs.jsonl with real per-agent tokens/cost/latency for this run (total $0.211878, 42.4% of the $0.50/run cap) -- CrewAI's in-process token-usage path, contrasted with n8n's post-hoc REST-API ingestion, discussed in Section 3.